

Teórica 9
Code Smells



NOVA SCHOOL OF SCIENCE & TECHNOLOGY

Engenharia de Software 2026/2027

Por que é que o programador trouxe ambientador para o escritório?

Anti-padrões ou Code Smells



Refactoring

É o processo de fazer alterações ao código para que os comportamentos externos dos padrões de desenho não sejam alterados, e a estrutura interna seja melhorada.



Anti-padrões ou Code Smells

Os padrões surgem no processo de desenho.

Os anti-padrões surgem no código.

Qual é o problema?

```
public class LibraryCatalog {
    private List<Book> books;
    // Constructor and other methods...
    public boolean checkoutBook(String bookTitle,
                                String patronName) {
        // Duplicate input validation code.
        if (bookTitle == null || bookTitle.isEmpty()) {
            System.out.println("Invalid book title.");
            return false;
        }
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the checkout logic...
        return true;
    }
```

```
    public boolean returnBook(String bookTitle, String patronName) {
        // Duplicate input validation code.
        if (bookTitle == null || bookTitle.isEmpty()) {
            System.out.println("Invalid book title.");
            return false;
        }
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the return logic...
        return true;
    }
}
```

Code Smell: Duplicated Code



Ocorre quando existem blocos de código que são semelhantes, mas que apenas apresentam pequenas diferenças em vários locais.

Problema:

- Quando é necessário fazer alterações, o código tem de ser actualizado em vários locais.
- Isto aplica-se quando se pretende adicionar funcionalidades, actualizar um algoritmo ou corrigir um bug.

Code Smell: Duplicated Code

Se só precisássemos de alterar o código num sítio, seria muito mais simples implementar a alteração.

Desta forma, também se reduz a probabilidade de nos esquecermos de algum pedaço de código que tenha de ser alterado.

Princípio D.R.Y., ou "Don't Repeat Yourself"

Os programas devem ser escritos de forma a realizar as mesmas tarefas com menos código

Same code found in multiple methods from the same class

```java
public class LibraryCatalog {
    private List<Book> books;
    // Constructor and other methods...
    public boolean checkoutBook(String bookTitle,
                                String patronName) {
        // Duplicate input validation code.
        if (bookTitle == null || bookTitle.isEmpty()) {
            System.out.println("Invalid book title.");
            return false;
        }
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the checkout logic...
        return true;
    }
```

```java
    public boolean returnBook(String bookTitle, String patronName) {
        // Duplicate input validation code.
        if (bookTitle == null || bookTitle.isEmpty()) {
            System.out.println("Invalid book title.");
            return false;
        }
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the return logic...
        return true;
    }
}
```

Extraia o método e invoque-o onde o código estava duplicado

```
public class LibraryCatalog {
    private List<Book> books;
    // Constructor and other methods...
    public boolean checkoutBook(String bookTitle,
                                String patronName) {
        // Duplicate input validation code.
        if (!checkTitle(bookTitle)) return false;
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the checkout logic...
        return true;
    }
```

```
    public boolean returnBook(String bookTitle, String patronName) {
        // Duplicate input validation code.
        if (!checkTitle(bookTitle)) return false;
        if (patronName == null || patronName.isEmpty()) {
            System.out.println("Invalid patron name.");
            return false;
        }
        // Rest of the return logic...
        return true;
    }
}
private boolean checkTitle(String booktTitle) {
    if (bookTitle == null || bookTitle.isEmpty()) {
        System.out.println("Invalid book title.");
        return false;
    }
}
```

Extraia o método e invoque-o onde o código estava duplicado

```
public class LibraryCatalog {
    private List<Book> books;
    // Constructor and other methods...
    public boolean checkoutBook(String bookTitle,
                                String patronName) {
        if (!checkTitle(bookTitle)) return false;
        if (!checkPatron(patronName)) return false;
        // Rest of the checkout logic...
        return true;
    }
    public boolean returnBook(String bookTitle,
                              String patronName) {
        if (!checkTitle(bookTitle)) return false;
        if (!checkPatron(patronName) return false;
        // Rest of the return logic...
        return true;
    }
}
```

```
private boolean checkTitle(String booktTitle) {
    if (bookTitle == null || bookTitle.isEmpty()) {
        System.out.println("Invalid book title.");
        return false;
    }
}
private boolean checkPatron(String patronName) {
    if (patronName == null || patronName.isEmpty()) {
        System.out.println("Invalid patron name.");
        return false;
    }
}
```

Extraia o método e invoque-o onde o código estava duplicado

```
public class LibraryCatalog {
    private List<Book> books;
    // Constructor and other methods...
    public boolean checkoutBook(String bookTitle,
                                String patronName) {
        if (!checkBook(bookTitle, patronName)) return false;
        // Rest of the checkout logic...
        return true;
    }
    public boolean returnBook(String bookTitle,
                              String patronName) {
        if (!checkBook(bookTitle, patronName)) return false;
        // Rest of the return logic...
        return true;
    }
}
```

```
private boolean checkTitle(String bookTitle) {
    if (bookTitle == null || bookTitle.isEmpty()) {
        System.out.println("Invalid book title.");
        return false;
    }
}
private boolean checkPatron(String patronName) {
    if (patronName == null || patronName.isEmpty()) {
        System.out.println("Invalid patron name.");
        return false;
    }
}
private boolean checkBook(String bookTitle, String patronName){
    return (checkTitle(bookTitle) && checkPatron(patronName));
}
```

Refactoring: Duplicated Code

Se o código repetido estiver em duas subclasses do mesmo nível:

- Extraia o método (Extract Method) para ambas as classes e mova os campos (Pull up Field) para utilizados no método extraído para ambas as classes
- Se o código repetido estiver dentro de um construtor, extraia o corpo do construtor (Pull up constructor body).
- Se o código duplicado for semelhante, mas não completamente idêntico, crie template method.
- Se dois métodos fizerem a mesma coisa, mas usarem algoritmos diferentes, selecione o melhor algoritmo e substitua-o (Substitute Algorithm).

Refactoring: Duplicated Code

Se o código duplicado estiver em classes diferentes:

- Se as classes não fizerem parte de uma hierarquia, extraia uma superclasse, a fim de manter o código duplicado.
- Se não fizer sentido criar uma superclasse, extraia a classe de um dos componentes e utilize a classe extraída no outro componente.

Refactoring: Duplicated Code

Se existir um número elevado de expressões condicionais que executam o mesmo código (diferindo apenas nas condições):

- Junte esses operadores numa única condição Consolidando a Expressão Condicional e Extraindo o Método de forma a colocar a condição num método separado com um nome fácil de perceber.

Refactoring: código duplicado

Se o mesmo código for executado em todos os ramos de uma expressão condicional:

- Coloque o código idêntico fora da árvore de condições usando Consolidar fragmentos condicionais duplicados.

Antes:

```
if (isSpecialDeal()) {
  total = price * 0.95;
  send();
} else {
  total = price * 0.98;
  send();
}
```

Depois:

```
if (isSpecialDeal())
  total = price * 0.95;
else
  total = price * 0.98;

send();
```

Qual é o problema?

```
public class Example {
  public void processOrder(Order order) {
    // Step 1: Validate the order
    if (order == null)
      throw new IllegalArgumentException("Order is null");

    if (order.getItems().isEmpty())
      throw new IllegalArgumentException("Order has no items");

    if (order.getTotal() <= 0)
      throw new IllegalArgumentException("Invalid order total");

    // Step 2: Calculate taxes and discounts
    double tax = order.getTotal() * 0.10;
    double discount = 0;

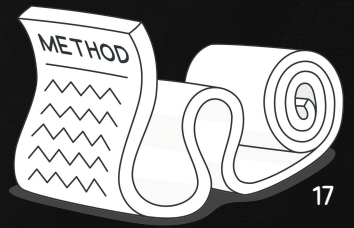
    if (order.getTotal() >= 100)
      discount = 10;

    // Step 3: Apply taxes and discounts
    order.setTotal(order.getTotal() + tax - discount);

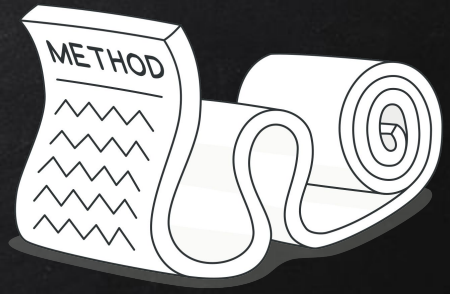
    // Step 4: Notify the customer
    EmailService.sendEmail(
         "Order Confirmation",
         "Your order has been processed.",
         order.getCustomer().getEmail()
    );

    // Step 5: Update the inventory
    for (OrderItem item : order.getItems()) {
        Product product = item.getProduct();
        int quantity = item.getQuantity();
        product.setQuantityInStock (
          product.getQuantityInStock() - quantity
        );
        ProductService.updateProduct(product);
    }

    // ...more code...
    // Step N: Other processing steps
    // Step N+1: Cleanup
    // ...more code...
  }
}
```



Code Smell: Long Method



O código não deve ter métodos longos

- São difíceis de compreender
- Pode ser um sinal de que o método é mais complexo do que deveria
- Tem mais lógica (funcionalidade, preocupações) do que deveria.
- Se for detectado um pedaço de código no método longo que:
  1. Pode existir por si só como um método e
  2. Consegue-se atribuir-lhe um nome informativo,

Então encontrou um bom candidato para factorizar parte do código num método independente (Extract Method)

Code Smell: Long Method

Mas... É preciso Cuidado!

- Pode ser difícil determinar se um método é muito longo.
  - Por exemplo, alguns métodos, como configurar uma interface de utilizador, podem ser naturalmente longos, mesmo que se concentrem apenas numa tarefa.
- Este code smell também pode depender da linguagem de programação do sistema.

Qual é o problema?

```
public class OrderProcessor {
    private List<Order> orders;
    private CustomerDatabase customerDB;
    private ProductDatabase productDB;
    private ShippingService shippingService;
    private PaymentService paymentService;

    // Constructor, getters, setters, and other methods...

    public void processOrder(Order order) {
        // Code to process an order...
    }
    public void calculateDiscounts() {
        // Code to calculate discounts...
    }
    public void sendConfirmationEmail(Customer customer) {
        // Code to send confirmation email...
    }
    public void generateInvoice(Order order) {
        // Code to generate an invoice...
    }
    public void updateInventory(Product product, int quantity) {
        // Code to update inventory...
    }
    // Many more methods...
    public void validateOrder(Order order) {
        // Code to validate an order...
    }
    public void applyTaxes(Order order) {
        // Code to apply taxes...
    }
    // Many more methods...
    // ... And more fields, methods, and responsibilities.
}
```




Code Smell: Large Class



As classes não devem ser muito grandes, geralmente denominadas:

- God Class
- Blob Class
- Black Hole Class

Ocorrem quando são necessárias mais responsabilidades e essas classes parecem ser o local apropriado para as implementar.

Com o tempo, estes «repositórios de responsabilidades» atraem ainda mais responsabilidades, e a classe fica cada vez maior, assumindo cada vez mais responsabilidades. É assim que surge o Large Class.

Code Smell: Large Class

Esse crescimento exige comentários extensos (um *desodorizante*) para documentar onde estão as funcionalidades no código da classe.

É útil detectar este code smell logo no início, pois quando a classe fica muito grande, pode ser difícil e demorado corrigi-lo.

> Em geral, as classes devem ter um objectivo explícito para serem coesas. Portanto, a classe deve fazer uma coisa bem.

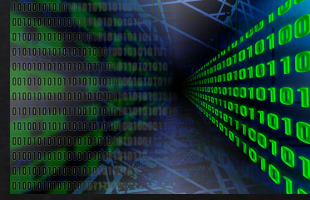
Se uma funcionalidade não for específica da classe, pode ser benéfico colocá-la noutro sítio (isto é, extrair o código para uma nova classe).

Refactoring: Large Class

Factorize a classe:

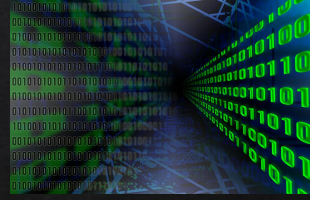
- Extract Class se for possível transferir parte do comportamento para outra classe
- Extract Subclass se for possível implementar parte do seu comportamento de maneiras diferentes ou for usada exporadicamente.
- Extract Interface se precisar de ter uma lista das operações e comportamentos que o cliente pode usar.
- Duplicate Observed Data se a classe for responsável pela interface gráfica. Pode tentar mover alguns dos dados e comportamentos para um objecto separado. Ao fazer isto, pode ser necessário armazenar cópias de alguns dados em dois locais e mantê-los consistentes.

Qual é o problema



```
public class Rectangle {
    private int height, width;
    public Rectangle(int height, int width) { … }
    public int getHeight() { … };
    public int getWidth(){ … };
}
```

Data Class



São classes demasiado pequenas.

Quando as classes contêm apenas dados, nenhuma funcionalidade real e apenas métodos *getters* e *setters* estão, provavelmente, na presença de uma Data Class.

Isto é um indicador de que a classe pode não ser uma boa abstracção ou sequer necessária.

- Existe código externo possa passar para a classe?
- Quais são as outras classes que manipulam estes dados?
- Alguns comportamentos estariam melhor se fossem implementados na Data Class?

Refactoring: Data Class

Adicionar comportamento interessante à Data Class (extraíndo-o de uma classe cliente):

```
public class Rectangle {
    private int height, width;
    public Rectangle(int height, int width) { … }
    public int getHeight() { … };
    public int getWidth(){ … };
    public int getArea() { return height * width; };
    public int getPerimeter() { return 2*height + 2 * width; };
}
```

Qual é o problema?

```
public class OrderProcessor {
    public void processOrder(String customerName, String customerEmail,
                             String productCode, int quantity) {
        // Code to process the order using customerName, customerEmail, productCode, and quantity...
    }

    public void sendOrderConfirmation(String customerName, String customerEmail) {
        // Code to send an order confirmation using customerName and customerEmail...
    }

    public void generateInvoice(String customerName, String customerEmail, double totalAmount) {
        // Code to generate an invoice using customerName, customerEmail, and totalAmount...
    }

    // More methods using the same set of data...
}
```



Code Smell: Data Clumps

"Data items tend to be like children; they enjoy hanging around in groups together. Often you'll see the same three or four data items together in lots of places: fields in a couple of classes, parameters in many method signatures. Bunches of data that hang around together really ought to be made into their own object." – Martin Fowler, 1999

Exemplo:

```
public void doSomething3D (int x, int y, int z) {
    ...
}
```



É melhor ter um objecto como parâmetro

```
public class Point3D {
     private int x;
     private int y;
     private int z;

     public int getX() { return x; }
     public int getY() { return y; }
     public int getZ() { return z; }

     public void setX(int newX) { x = newX; }
     public void setY(int newY) { y = newY; }
     public void setZ(int newZ) { z = newZ; }
}
```

Mas tenha cuidado para não criar apenas Data Classes (code smell). As classes devem fazer mais do que apenas armazenar dados.

O método `doSomething()` original, ou uma parte dele, pode ser adicionado à classe `Point3D` para evitar criar uma Data Class.

Qual é o problema?



Fonte: Marco Bianchetti, via unsplash

```
public class Product {
    // ...

    public void updateProductDetails(String name, String description, double price,
                                     String category, String manufacturer, int stockQuantity,
                                     int reorderThreshold,boolean isDiscounted, double discountPercentage){
        // Code to update product details using all these parameters...
    }
    // ...
}
```

Code Smell: Long Parameter List

Um método com uma longa lista de parâmetros pode ser difícil de usar, o que aumenta a probabilidade de algo correr mal.



Fonte: Marco Bianchetti, via unsplash

Long Parameter List: possíveis soluções

- Métodos com listas de parâmetros longas requerem comentários extensos para explicar o que cada parâmetro significa e o que representa (code smell)
- Evite uma "solução" comum para reduzir a quantidade de parâmetros: usar variáveis globais
  - na verdade, é quase uma espécie de code smell; dá origem a dependências globais!
- No entanto, se as listas de parâmetros não forem comentadas, pode ser necessário examinar perceber a implementação para perceber como devem ser utilizadas. Isto quebra o encapsulamento. (MAU!!!)

A melhor solução para este code smell é introduzir Parameter Objects.

Refactoring: Long Parameter List (exemplo)

Antes

```
public void updateProductDetails (
   String name, String description,
   double price, String category,
   String manufacturer, int stockQuantity,
   int reorderThreshold,
   boolean isDiscounted,
   double discountPercentage) {
     // Code to update product details using
     // all these parameters
}
```

Depois

```
public void updateProductDetails(Product product) {
 // Code to update product details using this
parameter...
}

public record Product(String name,
  String description, double price,
  String category, String manufacturer,
  int stockQuantity, int reorderThreshold,
  boolean isDiscounted,
  double discountPercentage) { … }
```

Agora podemos reutilizar este *record* sempre que precisarmos deste conjunto de informações.

Qual é o problema?



```
public class OrderProcessor {
      public void processOrder(Order order) {
          // Code to process the order...
      }
      public void sendOrderConfirmation(Order order) {
          // Code to send an order confirmation...
      }
      public void generateInvoice(Order order) {
          // Code to generate an invoice...
      }
      public void updateInventory(Order order) {
          // Code to update inventory...
      }
    public void applyDiscount(Order order) {
        // Code to apply discounts...
    }
```

Code Smell: Divergent Class



Refere-se a uma classe que assume várias responsabilidades ou sofre várias alterações por motivos que não estão diretamente relacionados com a sua responsabilidade principal.

Isso sugere que a classe se desviou do seu objetivo principal e está a lidar com multiplas responsabilidades.

As classes divergentes podem apresentar sinais de sobrecarga com métodos e responsabilidades que não pertencem logicamente a uma única classe.

Code Smell: Divergent Class



Relaciona-se com o code smell Large Class, onde classe grande tem muitas responsabilidades diferentes (ou seja, muitas abstracções, muitas preocupações).

- Large Class – concentre-se no tamanho e na complexidade da classe. Observe o número de atributos, métodos e linhas de código. Pode ter uma única responsabilidade, mas lidar com ela de maneira complexa e extensa.
- Divergent Class – Analise as responsabilidades da classe e como elas evoluíram. Identifique se a classe começou com uma responsabilidade bem definida, mas agora inclui métodos ou comportamentos que não estão relacionados com o seu propósito original. Isto sugere que a classe se desviou do seu objetivo principal.

A causa é *poor separation of concerns* (ou seja, má modularidade): isto é, uma causa comum de mudanças que divergem.

Separation of Concerns

As classes devem ter apenas um objetivo, ou seja, uma única preocupação

- reduz o número de razões pelas quais o código precisa de ser alterado
- reduz a variedade de alterações necessárias

Se uma classe está a ser alterada de várias formas, é um bom indicador de que as responsabilidades da classe devem ser divididas noutras classes, onde as responsabilidades devem ser extraídas para as classes apropriadas.

Separation of concerns resolve dois code smells: Large Class e Divergent Change.

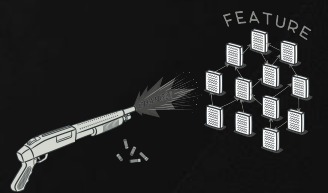
This stinks...

```
public class SavingsAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}

public class CheckingAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```

```
public class FixedDepositAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}

public class MoneyMarketAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```



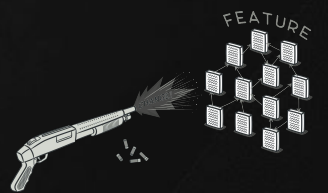
Neste exemplo o cálculo dos juros está repetido em cada class de contas. Se introduzirmos um novo requisito no cálculo, ou modificarmos a lógica dos cálculos, temos de alterar várias classes.

```
public class SavingsAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```

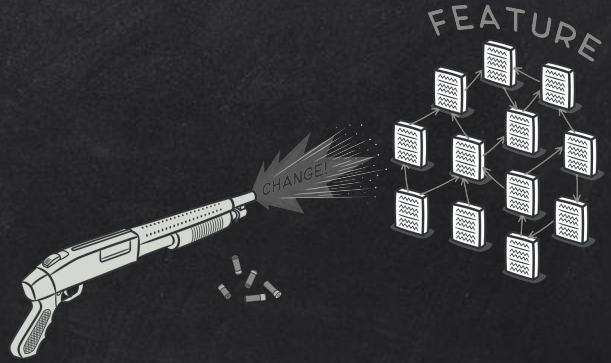
```
public class FixedDepositAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```

```
public class CheckingAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```

```
public class MoneyMarketAccount {
    // ... other methods ...
    public double calculateInterest() {
        // Code for interest calculation...
    }
}
```



Code Smell: Shotgun Surgery



Ocorre quando é necessário alterar um requisito e várias classes precisam de ser alteradas para acomodar a alteração.

Acontece quando se tenta adicionar novas funcionalidades, melhorar o código, resolver *bugs* ou mudar algoritmos.

Shotgun Surgery: Como evitar

- A Shotgun Surgery é normalmente resolvida mudando métodos de sítio.
- Se uma alteração exigir que se faça alterações em muitos métodos de diferentes classes, pode ser um indicador de que esses métodos devem ser consolidados numa ou duas classes.
  - 
    É preciso cuidado para não mover tantos métodos para uma classe e esta se venha a tornar numa Large Class (code smell).

Algumas alterações exigem Shotgun Surgery, independentemente de quão bem desenhado esteja o código. Por exemplo, alterar uma declaração de direitos de autor ou licenciamento exige que se alterem todos os ficheiros que incluam o texto de direitos de autor.

Exemplo: Suponha que existe uma classe Employee

```
public class Employee {
    private String name;
    private String address;
    private String position;

    // Constructor, getters, setters...

    public Employee (String name, String address, String position) {
        this.name = name;
        this.address = address;
        this.position = position;
    }
}
```

E também uma classe EmployeePrinter

```
// EmployeePrinter.java
public class EmployeePrinter {
    public void printEmployeeDetails(Employee employee) {
        System.out.println("Name: " + employee.getName());
        System.out.println("Address: " + employee.getAddress());
        System.out.println("Position: " + employee.getPosition());
    }
}
```

Finalmente, uma classe EmployeeDatabase

```
public class EmployeeDatabase {
    public void saveEmployee(Employee employee) {
        // Saving logic for name, address, position
        // Code to execute SQL...
    }
}
```

Que o pesadelo comece!

Agora, tente adicionar um número de telefone ao funcionário.

Fácil, certo? Bem...

```
public class Employee {
       private String name, address, position;
       private String phoneNumber;

       // Constructor, getters, setters...

       public Employee(String name, String address, String position) {
            this.name = name;
            this.address = address;
            this.position = position;
       }

      public String getPhoneNumber() {
            return phoneNumber;
      }
      public void setPhoneNumber(String phoneNumber) {
           this.phoneNumber = phoneNumber;
      }
} v
```

Desde que não se esqueça fazer a alteração nas muitas classes afectadas...

```
public class EmployeePrinter {
    public void printEmployeeDetails(Employee employee) {
        System.out.println("Name: " + employee.getName());
        System.out.println("Address: " + employee.getAddress());
        System.out.println("Position: " + employee.getPosition());
        System.out.println("Phone Number: " + employee.getPhoneNumber());
    }
}
```

Shotgun Surgery: Alterações em várias classes

```
public class EmployeeDatabase {
    public void saveEmployee(Employee employee) {
        // Saving logic for name, address, position
        // And now you have to add phoneNumber handling here as well
        String sql = "INSERT INTO employees (name, address, position, phone_number)”
                     + ”VALUES (?, ?, ?, ?)";
        // Code to execute SQL...
    }
}
```

Como podemos Resolver?

Encapsular os atributos do Employee num MAP para manter a estrutura da classe ao adicionar novos atributos

```
public class Employee {
  private Map<String, String> attributes;
  public Employee(String name, String address,
                  String position) {
    attributes = new HashMap<>();
    attributes.put("name", name);
    attributes.put("address", address);
    attributes.put("position", position);
  }

  // Getter and setter for attributes
  public String getAttribute(String key) {
    return attributes.get(key);
  }

  public void setAttribute(String key, String value) {
    attributes.put(key, value);
  }

  public Map<String, String> getAllAttributes() {
    return attributes;
  }
}
```

Agora, para adicionar um novo campo como phoneNumber, não precisamos de modificar a estrutura da classe Employee

Basta adicionar o novo campo, como uma nova entrada no `Map`, usando `setAttribute`.

```
Employee employee = new Employee(
                            "John Doe",
                            "123 Elm St.",
                            "Manager");
employee.setAttribute("phoneNumber", "555-1234");
```

Agora, quando precisarmos de imprimir os detalhes do Employee, seleciona-se automaticamente todos os atributos do mapa

```
public class EmployeePrinter {
  public void printEmployeeDetails(Employee employee) {
    for (Map.Entry<String, String> entry : Employee.getAllAttributes().entrySet())
      System.out.println(entry.getKey() + ": " + entry.getValue());
    }
  }
```

Também funciona para a EmployeeDatabase

```
public class EmployeeDatabase {
    public void saveEmployee(Employee employee) {
        StringBuilder sql = new StringBuilder("INSERT INTO employees (");
        StringBuilder values = new StringBuilder("VALUES (");
        for (String key : employee.getAllAttributes().keySet()) {
            sql.append(key).append(", ");
            values.append("?, ");
        }
        // Remove the last comma and space
        sql.setLength(sql.length() - 2);
        values.setLength(values.length() - 2);
        sql.append(") ").append(values).append(")");
        System.out.println("Executing SQL: " + sql.toString());
        // You would prepare a statement and execute this SQL with actual values in practice
    }
}
```

Não cheira nada bem...



```
public class Rectangle {
  private int height, width;
  public Rectangle(int height, int width) { … }
  public int getHeight() { … };
  public int getWidth(){ … };
}
public class GeometryCalculator {
…
  public int getRectanglePerimeter(Rectangle rectangle) {
    return rectangle.getHeight()*2 + rectangle.getWidth()*2;
  }
  public int getRectangleArea(Rectangle rectangle) {
    return rectangle.getHeight() * rectangle.getWidth();
  }
}
```

Code Smell: Feature Envy



Ocorre quando existe um método que está mais interessado nas funcionalidades públicas de outra classe do que nas suas.

Se um método ou classe está sempre a comunicar com um método de outra classe e parece que eles deveriam estar juntos, então é provável que seja verdade.

Refactoring: Feature Envy



Mova `getArea()` e `getPerimeter()` para a classe `Rectangle`:

```
public class Rectangle {
    private int height, width;
    public Rectangle(int height, int width) { … }
    public int getHeight() { … };
    public int getWidth(){ … };
    public int getArea() { return height * width; };
    public int getPerimeter() { return 2*height + 2 * width; };
}
```

Imagine que tem uma classe License que depende de uma classe Driver




```
public class License {
    private Driver driver;
    private int points;
    public License(Driver driver) { ... }
    public void setDriver(Driver driver) { this.driver = driver; }
    public int getPoints() { return points; }
    public void addPoints(int points) { this.points += points; }
    public String getSummary() {
       return driver.getTitle() + " " + driver.getFirstName() + " " +
              driver.getSurname() + Integer.toString(getPoints()) + " points.";
    }
}
```

Também tem uma classe Driver que depende de License

```
public class Driver {
   private String firstName;
   private String surname;
   private License license;
   public Driver(String firstName, String surname, License license){ ... }
   public String getSurname() { return surname; }
   public String getFirstName() { return firstName; }
   public RiskFactor getRiskFactor() {
      if (license.getPoints() > 3) return RiskFactor.HIGH;
      else if (license.getPoints() > return RiskFactor.MODERATE;
      else return RiskFactor.LOW;
   }
}
```

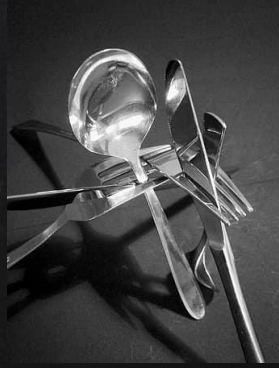


Isto força uma relação bi-direccional entre Driver e License




```
public class License {
   public String getSummary() {
       return driver.getTitle() + " " + driver.getFirstName() + " " +
              driver.getSurname() + Integer.toString(getPoints()) + " points.";
    }
}
public class Driver {
   public RiskFactor getRiskFactor() {
      if (license.getPoints() > 3) return RiskFactor.HIGH;
      else if (license.getPoints() > return RiskFactor.MODERATE;
      else return RiskFactor.LOW;
   }
}
```

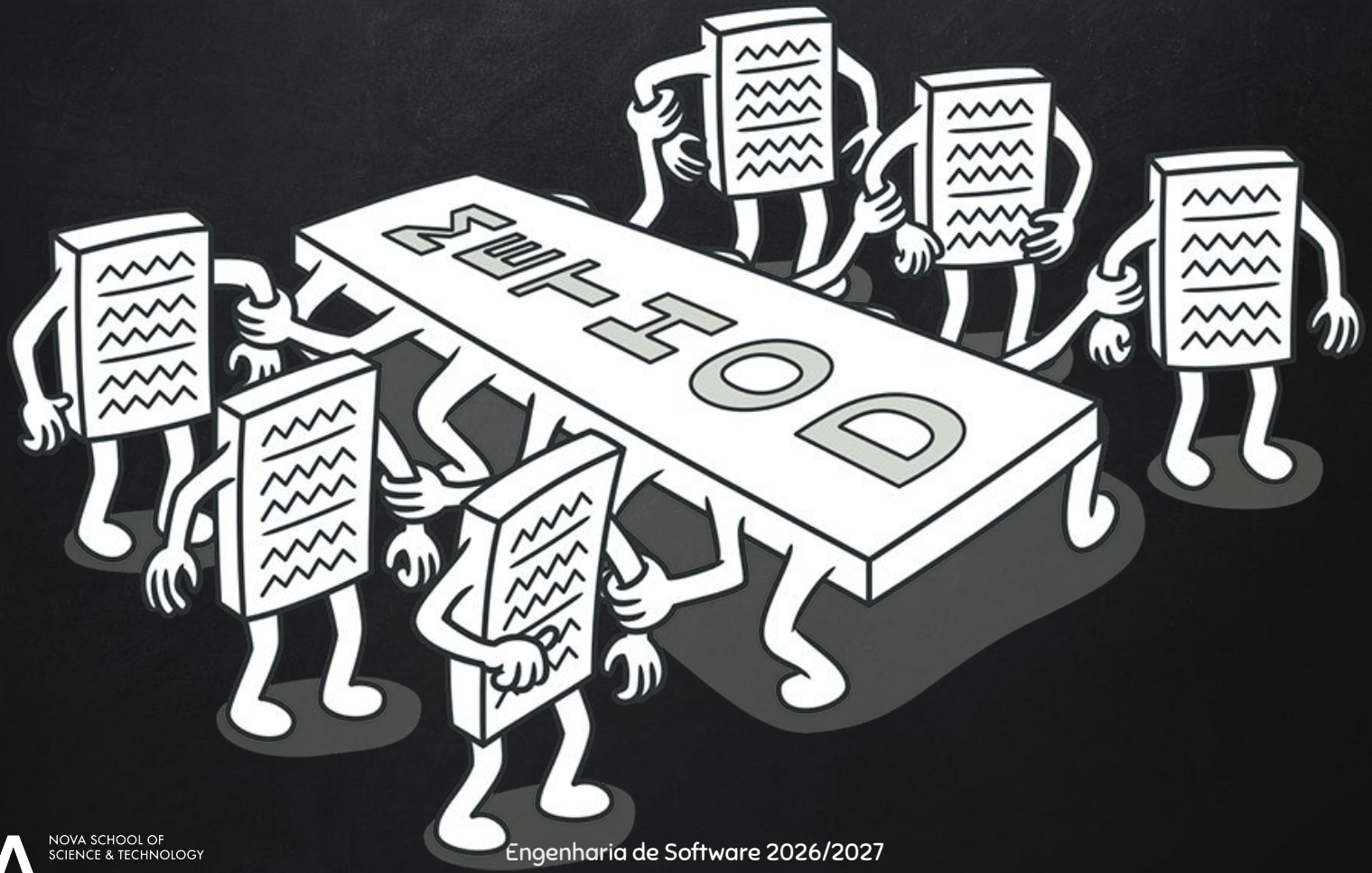
Code Smell: Inappropriate Intimacy



Ocorre quando duas classes dependem excessivamente uma da outra por meio de comunicação bidirecional.

Se duas classes estiverem intimamente acopladas, de forma a que um método de uma classe chama métodos da outra e vice-versa, então provavelmente será necessário remover esse ciclo. (cuidado, nem sempre é algo mau, por exemplo, GUIs)

Os métodos devem ser separados para que ambas as classes utilizem outra classe. No mínimo, isto torna a comunicação unidirecional e deve criar um acoplamento mais flexível.


SCRUM

Estas operações estão em classes trocadas! Mova-as para onde os dados estão disponíveis!




```
public class License {
   public String getSummary() {
       return driver.getTitle() + " " + driver.getFirstName() + " " +
              driver.getSurname() + Integer.toString(getPoints()) + " points.";
    }
}
public class Driver {
   public RiskFactor getRiskFactor() {
      if (license.getPoints() > 3) return RiskFactor.HIGH;
      else if (license.getPoints() > return RiskFactor.MODERATE;
      else return RiskFactor.LOW;
   }
}
```

getSummary não devia estar em License, já que usa informação de Driver. Sem isto, License não necessita de conhecer Driver

```
public class License {
    private Driver driver;
    private int points;
    public License(Driver driver) { ... }
    public void setDriver(Driver driver) { this.driver = driver; }
    public int getPoints() { return points; }
    public void addPoints(int points) { this.points += points; }
    public String getSummary() {
        return driver.getTitle() + " " + driver.getFirstName() + " " +
               driver.getSurname() + Integer.toString(getPoints()) + " points.";
    }
}
```



Muito mais simples

```
public class License {
    private int points;
    public License() { ... }
    public int getPoints() { return points; }
    public void addPoints(int points) { this.points += points; }
}
```



getSummary fica melhor em Driver. getRiskFactor ainda usa bastantes detalhes de License!

```
public class Driver {
   ...
   private License license;
   ...
   public RiskFactor getRiskFactor() {
      if (license.getPoints() > 3) return RiskFactor.HIGH;
      else if (license.getPoints() > return RiskFactor.MODERATE;
      else return RiskFactor.LOW;
   }
   public String getSummary() {
       return getTitle() + " " + getFirstName() + " " + getSurname() +
              Integer.toString(license.getPoints()) + " points.";
   }
}
```



Delegue getRiskFactor() na classe License!

```
public class Driver {
   ...
   private License license;
   ...
   public RiskFactor getRiskFactor() {
      return license.getRiskFactor();
   }
   public String getSummary() {
       return getTitle() + " " + getFirstName() + " " + getSurname() +
              Integer.toString(license.getPoints()) + " points.";
   }
}
```



Muito melhor!

```
public class License {
    private int points;
    public License() { ... }
    public int getPoints() { return points; }
    public RiskFactor getRiskFactor() {
        if (getPoints() > 3) return RiskFactor.HIGH;
        else if (getPoints() > return RiskFactor.MODERATE;
        else return RiskFactor.LOW;
   }
}
```



Qual é o Problema?

```
public class Appointment {
  private String title;
  private int year, int month;
  Private int day, hour, minute;

  public Appointment(String title,
      int year, int month, int day,
      int hour, int minute) {
    this.title = title;
    this.year = year;
    this.month = month;
    this.day = day;
    this.hour = hour;
    this.minute = minute;
  }
```

```
public boolean isUpcoming(int currentYear, int currentMonth, int currentDay,
                          int currentHour, int currentMinute) {
    if (year > currentYear)
      return true;
    if (year == currentYear && month > currentMonth)
      return true;
    if (year == currentYear && month == currentMonth && day > currentDay)
      return true;
    if (year == currentYear && month == currentMonth
        && day == currentDay && hour > currentHour)
      return true;

    return year == currentYear && month == currentMonth &&
           day == currentDay && hour == currentHour
           && minute > currentMinute;
  }
}
```




Qual é o Problema?

```
public class Appointment {
  private String title;
  private int year, int month;
  Private int day, hour, minute;

  public Appointment(String title,
      int year, int month, int day,
      int hour, int minute) {
    this.title = title;
    this.year = year;
    this.month = month;
    this.day = day;
    this.hour = hour;
    this.minute = minute;
  }
```

```
public boolean isUpcoming(int currentYear, int currentMonth, int currentDay,
                          int currentHour, int currentMinute) {
    if (year > currentYear)
      return true;
    if (year == currentYear && month > currentMonth)
      return true;
    if (year == currentYear && month == currentMonth && day > currentDay)
      return true;
    if (year == currentYear && month == currentMonth
        && day == currentDay && hour > currentHour)
      return true;

    return year == currentYear && month == currentMonth &&
           day == currentDay && hour == currentHour
           && minute > currentMinute;
  }
}
```




Code Smell: Primitive Obsession



Ocorre quando se depende demasiado do uso de tipos built-in (ou primitivos como ints, longs, floats ou strings)

- Apesar de necessários, apenas devem ser utilizados nos níveis mais baixos de código.
  - E.g., a utilização de Strings em demasia pode não ser uma boa abstracção: podemos aproveitar para desenhar classes compatíveis com a semântica das strings (o que significam).
- O uso excessivo de tipos primitivos ocorre quando as abstracções não são identificadas e as não são definidas classes adequadas.

Refactoring: Primitive Obsession
Usar uma abstracção adequada para data/hora!

```
public class Appointment {
  private String title;
  private LocalDateTime dateTime ;

  public Appointment (String title,
            int year,int month,
            int day, int hour, int minute) {
    this.title = title;
    this.dateTime = LocalDateTime.of (
         year, month, day, hour, minute);
}
```

```
public boolean isUpcoming(int currentYear,
        int currentMonth, int currentDay,
        int currentHour, int currentMinute) {

LocalDateTime other = LocalDateTime.of (
                        currentYear, currentMonth,
                        currentDay, currentHour,
                        currentMinute);

  return this.isBefore(other);
  }
}
```

Já que aqui estamos, porque não resolver as Long Parameter List e os Data Clumps em Appointment?

```
public class Appointment {
        private String title;
        private LocalDateTime dateTime ;

        public Appointment(LocalDateTime dateTime) {
                this.title = title;
                this.dateTime = dateTime);
        }
       public boolean isUpcoming(LocalDateTime other) {
                return this.isBefore(other);
        }
 }
```

Code Smell: Switch statements



Ocorre quando blocos `switch` estão espalhados pelo código.

Se um `switch` for alterado, todos os outros também devem actualizados.

Embora possa existir a necessidade de instruções `if/else` longas, às vezes as instruções `switch` podem ser resolvidas de uma forma mais inteligente.

Por exemplo, se as condições estiverem a fazer a verificação de tipos, existe uma forma melhor de o fazer. Pode ser possível reduzir as condições a um desenho que utilize polimorfismo.

Se as condições estiverem a fazer a verificação de tipos, existe uma forma melhor de o fazer.

```
public class Animal {
   private final static int DOG = 0;
   private final static int CAT = 1;
   private int type;
   …

  public void say() {
      switch (type) {
        case DOG -> System.out.println(“Woof!”);
        case CAT -> System.out.println(“Meow!”);
      }
   }
}
```

Pode ser possível reduzir as condições a um desenho que utilize polimorfismo.

Antes:

```
public class Animal {
   private final static int DOG = 0;
   private final static int CAT = 1;
   private int type;
   …

  public void say() {
      switch (type) {
        case DOG -> System.out.println(“Woof!”);
        case CAT -> System.out.println(“Meow!”);
      }
   }
}
```

Depois:

```
public interface Animal {
   void say();
}

public class Dog implements Animal {
   …
   public void say() {
         System.out.println(“Woof!”);
   }
}

public class Cat implements Animal {
   …
   public void say() {
         System.out.println(“Meow!”);
   }
}
```

Imagine que está a desenvolver um jogo de estratégia medieval em que tem três tipos de soldados e mais nada. O que está mal?

```
public class Animal {
   private int health;
}
public class Soldier extends Animal {
   private int attackPower;
   ...
}
public class Archer extends Soldier { ... }
public class ManAtArms extends Soldier { ... }
public class Knight extends Soldier { ... }
```

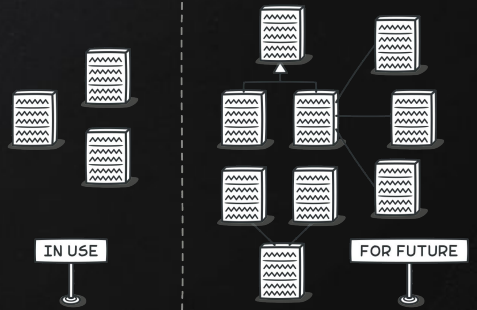


Code Smell: Speculative generality

Ocorre quando cria uma superclasse, interface ou outro código de que não necessita agora, mas pode ser útil um dia. Esta prática introduz generalidade, que pode não ajudar, mas antes simplesmente complicar a solução.

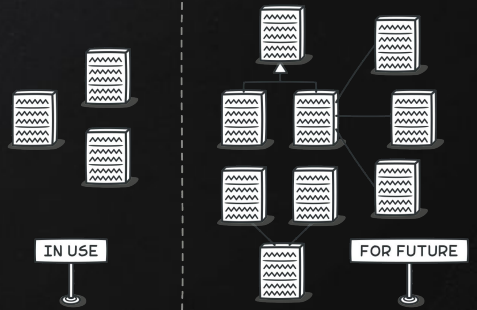
Lembre-se:

No desenvolvimento ágil, é melhor fazer o chamado *Just in Time Design*. Isto significa que deve haver apenas o design essencial para os requisitos existentes para implementar a iteração actual de um sistema que tem de funcionar agora.



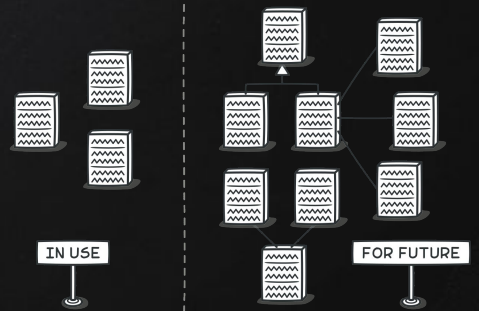
Não há animais no jogo! Porque é que os está a modelar, a pensar numa versão do jogo que nem sabe se algum dia vai implementar, para introduzir a noção de caça?

```
public class Animal {
   private int health;
}
public class Soldier extends Animal {
   private int attackPower;
   ...
}
public class Archer extends Soldier { ... }
public class ManAtArms extends Soldier { ... }
public class Knight extends Soldier { ... }
```



Não complique sem necessidade. Se e quando precisar de animais, logo pensa em como os implementar no jogo!

```
public class Soldier {
   private int health, attackPower;
   ...
}
public class Archer extends Soldier { ... }
public class ManAtArms extends Soldier { ... }
public class Knight extends Soldier { ... }
```



Code Smell: Speculative Generality

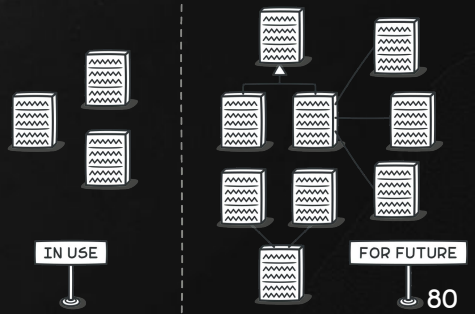
Ocorre quando se cria uma superclasse, interface ou código que não é necessário por enquanto, mas que, um dia, pode vir a ser útil.

Esta prática introduz uma generalidade que pode não ajudar e até corremos o risco de "over-engineer".

Lembre-se:

No desenvolvimento Agile, é melhor praticar *Just in Time Design*:

Deve se deve implementar apenas o que é suficiente para responder aos requisitos de uma iteração para ter um sistema funcional.



Deodorants: Comments

Não, não estamos a dizer que os comentários são um code smell 😄

No entanto... Às vezes, os comentários são usados como um *deodorant*.

"It's surprising how often you look at thickly commented code and notice that the comments are there because the code is bad" – Martin Fowler

Utilizar muitos comentários para explicar código complicado pode indicar o encobrimento de mau desenho ou estilo

- Por vezes, extrair novos métodos (com nomes descritivos) repletos de comentários resolve o problema.
  - Em muitas situações, os comentários são dicas para os nomes dos novos métodos.

Deodorants: Comments

- Pode ser necessário fazer alguns comentários se a linguagem de programação não suportar diretamente alguns princípios de desenho
  - E.g., as versões anteriores do Java não tinham tipos genéricos. Os programadores precisavam de comentários para explicar os tipos de conversão.
- Então, quando devemos usar comentários? (além do javadoc)
  - Quando não sabemos o que fazer. Além de descrever o que está a acontecer, os comentários indicam áreas de incerteza
  - Para explicar o porquê de uma decisão
  - Lembretes TO DO – para indicar algo ainda não está implementado.
- Repare que, se uma secção de código for alterada e, por este motivo for necessário actualizar outra secção, poderá ser um indicador de mau código.

Qual é o Problema?




```
public class Vehicle {
  public void startEngine() {
    // Common implementation to start the engine.
  }

  public void accelerate() {
    // Common implementation to accelerate.
  }

  public void brake() {
    // Common implementation to brake.
  }
}
```

```
public class Car extends Vehicle {
  // Car-specific methods and data...
}

public class Bicycle extends Vehicle {
  // Bicycle-specific methods and data...
}
```

Code Smell: Refused Bequest



Ocorre quando uma subclasse herda comportamentos mas não precisa deles.

Se uma superclasse declara um comportamento comum entre subclasses e as subclasses estão a herdar coisas que não precisam ou não usam, então podem não ser subclasses apropriadas para a superclasse.

Soluções possíveis:

- Pode fazer mais sentido para uma stand-alone class (e.g. `Bicycle` deve existir por si só).
- Ou os comportamentos não devem ser definidos na superclasse.
- Se apenas algumas subclasses os utilizam, então pode ser melhor definir esses comportamentos apenas nas subclasses.

Qual é o Problema?

```
public class Employee {
    private String name;
    private Department department;
    public Employee(String name, Department department) {
        this.name = name;
        this.department = department;
    }
    public String getManagerEmail() {
        return department.getManager().getEmail();
    }
}
```

Qual é o Problema?

```
public class Employee {
    private String name;
    private Department department;
    public Employee(String name, Department department) {
        this.name = name;
        this.department = department;
    }
    public String getManagerEmail() {
        return department.getManager().getEmail();
    }
}
```

Code Smell: Message Chains

Ocorre quando o código tem longas cadeias de mensagens onde se invoca um método, recebe-se um objecto e invoca-se novamente um método sobre o objecto devolvido anteriormente.

Exemplo: `a.getB().getC().doSomething();`

O exemplo anterior é uma cadeia de mensagens inadequada, que pode causar rigidez ou complexidade no desenho e tornar o código mais difícil de testar de forma independente.

Refactoring: Message Chains (Comunicar apenas com os objectos vizinhos)

```
public class Employee {
  private String name;
  private Department department;
  public Employee(String name, Department department) {
    this.name = name;
    this.department = department;
  }

  public String getManagerEmail() {
    // Naturally Department now has to offer its Manager's email address.
    return department.getManagerEmail();
  }
}
```

Code Smells: resumo

Bloaters: métodos e classes que chegaram a proporções em que é difícil de trabalhar.

Long method, Large Class, Primitive Obsession, Long Parameter List, Data Clumps

Abusers: aplicação incompleta ou incorreta dos princípios da programação orientada a objetos.

Switch Statement

Change Preventers: quando alterar um pedaço de código implica fazer alterações noutros sítios.

Divergent Class, Shotgun Surgery

Code Smells: Resumo

Dispensables: Algo inútil e desnecessário cuja ausência tornaria o código mais limpo, eficiente e fácil de entender.

Duplicate Code, Data Class, Speculative Generality

Couplers: contribuem para o acoplamento excessivo entre classes ou mostram o que acontece se o acoplamento for substituído por delegação excessiva.

Feature Envy, Inappropriate Intimacy, Message Chains

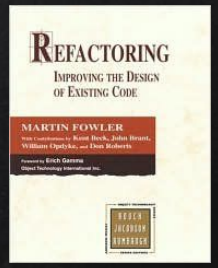
Porque é que o programador trouxe um ambientador para o escritório?

Porque o código cheirava mal!

Referências

Refactoring, Improving the Design of Existing Code.

Martin Fowler, with Kent Beck, John Brant, William Opdyke, Don Roberts. Addison-Wesley, 1999.



Refactoring Guru. https://refactoring.guru/

